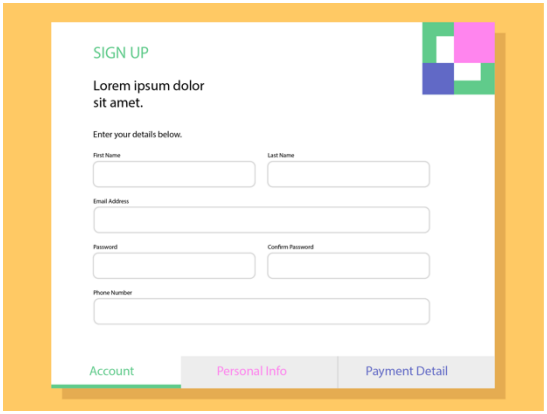
Form Elements

What are Forms?
In HTML, forms are interactive elements that allow users to input and submit data on a web page. They typically include various input fields like text boxes, radio buttons, checkboxes, and buttons, enabling users to send information to a server for processing.



Alt text: *Form*

Create a Form
Inside the <body> section, add a <form> element. The action and the method are the attributes of form that we will discuss later.

```
<form action="/submit" method="post">
    <!-- Form content will go here -->
</form>
```

Form Elements & Attributes
Form elements in HTML are the building blocks used to create interactive forms on web pages. These elements allow users to input and submit data. Here are some common form elements:

a. <form>
b. <input>
c. <textarea>
d. <select>
e. <label>

a. <form>-

The container for all form elements. It defines the start and end of the form.

Syntax

```
<body>
    <form [attributes]>
      <!-- Form content goes here -->
    </form>
</body>
```

Attribute	Example	Description

action	`<form action="/submit_form" method="post">`
	`<!-- Form content -->`
	`</form>`
		Specifies the URL where the form data will be sent when the form is submitted.

method	`<form action="/submit_form" method="post">`
	`<!-- Form content -->`
	`</form>`
		Defines the HTTP method used to send the form data. The two most common methods are GET and POST.

target	`<form action="/submit_form" method="post" target="_blank">`
	`<!-- Form content goes here -->`
	`</form>`
		Specifies where to open the form response. Common values include _blank (open in a new tab or window) and _self (open in the same frame or tab).

autocomplete	`<form action="/submit_form" method="post" autocomplete="off">`
	`<!-- Form content goes here -->`
	`</form>`
		Enables or disables the browser's autocomplete feature for the form. Values can be on or off.

novalidate	`<form action="/submit_form" method="post" autocomplete="off" >`
	`<!-- Form content goes here -->`
	`</form>`
		disables browser validation and is useful when you want to perform custom validation using JavaScript and want to prevent the default browser validation.

GET Method:
The GET method is used to request data from a specified resource. When you use a web browser to visit a website, the browser typically sends a GET request to the server to retrieve the specified web page or resource. The data is sent in the URL as a query string, visible in the address bar of the browser.

Example:
Suppose you have a simple webpage that displays information about a product based on its ID. The URL might look something like this:

https: //example. com/product?id=123

In this URL:

- **https: //example. com** is the base URL.
- **product** is the endpoint or route.
- **id=123** is the query string, specifying the ID of the product to retrieve.

POST Method:

The POST method is used to submit data to be processed to a specified resource. Unlike the GET method, data sent with POST is not visible in the URL, making it suitable for sending sensitive information like passwords.

Example:
Consider a registration form on a website where users can sign up for an account. When a user fills out the registration form and submits it, the browser sends a POST request to the server with the user's information included in the request body.

b. <input>

Input element is the most important element of the form. Users can send input in any form using this element.

Attributes of <input> tag:

Attribute	Example	Description
type	`<input type="text">`	Specifies the type of input control to be created. Common values include "text", "password", "checkbox", "radio", "number", "email", etc.
name	`<input type="text" name="username">`	The name is used to identify the input when the form is submitted.
value	`<input type="text" value="Default Text">`	Sets the initial value of the input. For text inputs, it represents the default text; for checkboxes and radio buttons, it represents the default checked state.
placeholder	`<input type="text" placeholder="Enter your name">`	Provides a short hint or example text that is displayed in the input field before the user enters a value. It disappears when the user starts typing.
required	`<input type="text" id="username" name="username" required>`	Useful for ensuring that essential information is provided by users before processing form submissions

HTML Code	Output
`<body>` `  <!-- Simple form with a username input, label, and submit button -->` `  <form action="/submit" method="post">` `  <label for="username">Name:  </label>` `  <input type="text" id="username" name="username" placeholder="Enter your name here" required>` `     ` `  <input type="submit" value="Submit">` `  </form>` `</body>`	Name: Enter your name here Submit

This HTML code creates a simple form for capturing a user's name. A form is defined using the <form> element with an action attribute set to "/submit" and a method attribute set to "post," indicating that the form data will be sent to the "/submit" endpoint using the HTTP POST method.

The form contains a label, associated with an input field using the "for" attribute, prompting the user to enter their name. The input field has an "id" of "username," a "name" attribute set to "username," a placeholder indicating where to enter the name, and the "required" attribute, ensuring that the field must be filled out before submitting the form. Finally, a submit button is included with the text "Submit."

c. <textarea>-

It is used to create a multi-line text input area where users can enter and edit longer blocks of text.

Attributes of <textarea> tag:

Attributes	Example	Description
rows	`<textarea rows="4" >Enter your text here...</textarea>`	It specifies the visible number of lines in the textarea.

cols	`<textarea cols="50">Enter your text here...</textarea>`	It specifies the visible width of the textarea in terms of characters.
placeholder	`<textarea placeholder="Type your message here..."></textarea>`	It can be used to provide a hint or example text within the textarea.
name	`<textarea name="userMessage" rows="4" cols="50" ></textarea>`	It is used to identify the textarea when the form is submitted.

Html Code	Output
`<body>` `  <form>` `  <textarea name="comments" rows="4" cols="50" placeholder="Type your comments here..."> ` `  </textarea>` `   ` `  <input type="submit" value="Submit">` `  </form>` `  </body>`	

This HTML code creates a form with a textarea input field where users can type comments. The textarea has attributes for specifying the name, rows, cols, and a placeholder text. Below the textarea, there's a submit button. When the form is submitted, the entered comments are sent to the server.

d. <select>-

It is used to create a drop-down list within a form, allowing users to choose from a set of predefined options.

Syntax

```
<select [attribute="value"]>
  <!-- Options go here -->
</select>
```

Attributes of <select> Tag:

Attributes	Example	Description
name	`<select name="country">`	The name is used to identify the select control when the form is submitted
size	`<select size="3">`	Specifies the number of visible options in a drop-down list.
multiple	`<select multiple>`	allows users to select multiple options by holding down the Ctrl key while clicking
disabled	`<select disabled>`	Disables the select element, preventing user interaction.
required	`<select required>`	Specifies that the user must select an option before submitting the form.

Html Code	Output

`<body>`
`  <select id="country" name="country">`
`  <option value="usa">United States</option>`
`  <option value="uk">United Kingdom</option>`
`  <option value="canada">Canada</option>`
`  </select>`
`</body>`	

The HTML code creates a dropdown menu (<select>) with options for selecting a country. The options are United States, United Kingdom, and Canada. Each option has a corresponding value (usa, uk, canada) that can be sent to a server when selected.

e. <label>-

It is used to associate a label with a form control, providing a more accessible and user-friendly interface.

Syntax:

```
<label> content</label>
```

Attributes of <label> tag:

For- It defines the form element that a label is describing.

Form- It defines a form to which a label belongs

autofocus- Specifies that the label should automatically receive focus when the page loads.

Attributes	Example	Description
For	`<label for="username">Username:</label>`	Associates the <label> with a specific form control using the id attribute of the form control.
Form	`<label for="email" form="myForm">Email:</label>`	Specifies the form to which the labelled control belongs.
autofocus	`<label for="search" autofocus>Search:</label>`	Specifies that the label should automatically receive focus when the page loads.

HTML Code	Output
`<label for="password">Password:</label>` `<input type="password" id="password" name="password" required>`	Password:

This code creates a labelled password input field in an HTML form that requires the user to enter a password before submitting the form, and it masks the characters entered for security.

Let us understand the code line by line.

```
<label for="password">Password:</label>
```

- **<label>**: This is an HTML element used to define a label for an input element.
- **for="password"**: This attribute associates the label with the input element with the id "password". So when you click on the label, it focuses on the input field.
- "Password:": This is the text that will be displayed as the label for the input field.

```
<input type="password" id="password" name="password" required>
```

- **<input>**: This is an HTML element used to create an input field.
- **type="password"**: This attribute specifies that the input field is for entering passwords. The characters entered into this field are typically masked for security reasons (usually as asterisks or dots).
- **id="password"**: This attribute assigns an identifier to the input field. It should be unique within the HTML document.
- **name="password"**: This attribute specifies the name of the input field. When the form is submitted, this name is used to identify the data sent to the server.
- **required**: This attribute specifies that the input field must be filled out before submitting the form. If the input field is empty when the form is submitted, an error message will be displayed prompting the user to fill it out.